Atividade Prática de Fixação: Especificadores de Acesso na Linguagem C

📌 **Objetivo:** Ajudar os alunos a **fixarem o uso de especificadores de acesso** e **qualificadores de variáveis** na **linguagem C**. A atividade foca no **uso de `extern`, `static`, `auto`, `register`, `const` e `volatile`** e como eles afetam a visibilidade, a persistência e o comportamento das variáveis no código. Os alunos deverão escrever um **script de C**, utilizando esses qualificadores e praticando **boas práticas de programação**, incluindo **comentários** e **nomenclatura clara**.

📌 **Formato: Atividade Individual**
📌 **Duração: 2 horas**
📌 **Ferramentas: Compilador de C** (Dev C++, Code::Blocks, ou IDE online como replit.com)
📌 **Entrega: Script em C** com explicações, boas práticas e cabeçalho formatado em **Bloco de Notas**.

📖 Contextualização e Situação Problema:

Você foi contratado para desenvolver um programa que simula o controle de estoque em uma loja de **produtos eletrônicos**. O programa deve **registrar entradas e saídas de produtos**, mantendo controle da **quantidade em estoque** e **gerar relatórios**.

A ideia dessa atividade é utilizar **modificadores de acesso** (`extern`, `static`, `auto`, `register`, `const` e `volatile`) para controlar o comportamento de variáveis e otimizar o código. Você também vai usar boas práticas de programação para tornar seu código mais legível e eficiente.

📋 Passo a Passo de Execução:

1. Preparação do Ambiente (10 minutos):

- Abra seu **compilador de C** (Dev C++, Code::Blocks ou IDE online).
- Crie um **novo arquivo** de código C e salve com o nome `controle_estoque.c`.

2. Estrutura Básica do Programa (20 minutos):

Escreva a estrutura básica de um programa em C, incluindo **declaração de variáveis**, **entrada e saída de dados** e **comandos de controle de estoque**.

Código Inicial:

C/C++

```
#include <stdio.h>  // Biblioteca para entrada e saída de dados

// Declaração de variáveis globais
extern int quantidadeTotal;  // Variável global definida em outro
arquivo

// Variável estática para controle de estoque
static int quantidadeProduto = 0;

// Função para atualizar o estoque de um produto
void atualizarEstoque(int quantidade) {
    quantidadeProduto += quantidade;  // Atualiza a quantidade do
produto
    printf("Estoque atualizado! Quantidade: %d\n",
quantidadeProduto);
}

int main() {
    // Definindo o preço do produto como constante
    const float precoProduto = 50.0;

    // Variáveis locais
    int quantidadeEntrada, quantidadeSaida;

    // Entrada de dados
    printf("Digite a quantidade de produtos a serem adicionados ao
estoque: ");
    scanf("%d", &quantidadeEntrada);

    // Atualiza o estoque com a quantidade de entrada
    atualizarEstoque(quantidadeEntrada);

    // Entrada de saída de dados
    printf("Digite a quantidade de produtos a serem retirados do
estoque: ");
```

```
    scanf("%d", &quantidadeSaida);

    // Atualiza o estoque com a quantidade de saída
    if (quantidadeProduto >= quantidadeSaida) {
        atualizarEstoque(-quantidadeSaida);
    } else {
        printf("Estoque insuficiente para a retirada.\n");
    }

    // Exibe o relatório final
    printf("Relatório final:\n");
    printf("Preço do produto: %.2f\n", precoProduto);
    printf("Quantidade em estoque: %d\n", quantidadeProduto);

    return 0;
}
```

3. Explicação dos Modificadores de Acesso (30 minutos):

1. **`extern`**:
    - A variável `quantidadeTotal` foi declarada com `extern`, o que significa que sua definição será em outro arquivo.
    - **Objetivo:** Demonstrar o uso de variáveis globais compartilhadas entre arquivos.
2. **`static`**:
    - A variável `quantidadeProduto` foi declarada com `static`, o que significa que ela é **visível apenas dentro da função `atualizarEstoque`** ou dentro do arquivo, impedindo o acesso de outras partes do código fora deste arquivo.
    - **Objetivo:** Controlar a visibilidade da variável ao longo do programa e garantir que ela mantenha o valor entre chamadas.
3. **`const`**:

- A constante `precoProduto` foi definida com `const`, o que significa que o valor não pode ser alterado depois de atribuído.
- **Objetivo:** Garantir que o preço do produto não seja modificado acidentalmente.

4. Otimização com `register` (20 minutos):

Adicionar o uso de **`register`** para otimizar variáveis frequentemente usadas, como no caso de um contador de loop.

Código de exemplo com `register`:

```
C/C++
void processarEstoque() {
    register int i;
    for (i = 0; i < quantidadeProduto; i++) {
        printf("Produto %d em estoque.\n", i + 1);
    }
}
```

- **Explicação:** O modificador `register` sugere que a variável `i` seja armazenada em um **registrador de CPU**, o que pode melhorar o desempenho quando a variável é usada frequentemente.

5. Uso de `volatile` (20 minutos):

Adicionar o modificador `volatile` em uma variável que pode ser alterada por outro processo, como em sistemas embarcados ou multitarefa.

Código de exemplo com `volatile`:

```
C/C++
volatile int contadorDeErros;

void verificarErros() {
    while (contadorDeErros == 0) {
        // Espera até que o contador de erros seja alterado
```

```
    }
    printf("Erros detectados!\n");
}
```

- **Explicação:** O `volatile` garante que a variável `contadorDeErros` não seja otimizada pelo compilador, sendo lida sempre da memória.

6. Boas Práticas de Programação (20 minutos):

- **Comentários:** Comente o código explicando a função de cada parte, como variáveis, funções e os modificadores de acesso.
- **Nomes significativos:** Use nomes de variáveis e funções que representem claramente o que elas fazem.
- **Indentação:** Mantenha a indentação do código para facilitar a leitura e manutenção.

Exemplo de boas práticas:

```C/C++
// Função para atualizar o estoque
// Recebe uma quantidade (positiva ou negativa) para adicionar ou remover do estoque
void atualizarEstoque(int quantidade) {
    quantidadeProduto += quantidade;  // Atualiza a quantidade do produto
    printf("Estoque atualizado! Quantidade: %d\n", quantidadeProduto);
}
```

7. Teste e Validação (20 minutos):

Execute o programa com diferentes entradas de dados (quantidade de entrada e saída de produtos) e verifique se o programa:

- Atualiza corretamente o estoque.

- Impede saídas quando o estoque for insuficiente.
- Exibe corretamente o relatório.

📄 Formato de Entrega:

1. **Cabeçalho no Script:** O arquivo deve conter um cabeçalho no início do código, explicando o nome do programa, a versão, o autor, a data de entrega e a descrição.

Cabeçalho Exemplo:

```C/C++
// Programa: Controle de Estoque
// Versão: 1.0
// Autor: [Nome do Estudante]
// Data: 25/03/2025
// Descrição: Programa de controle de entrada e saída de produtos em estoque, utilizando modificadores de acesso e boas práticas de programação.
```

2. **Código Completo:** O script em C deve ser enviado no formato **.txt** (Bloco de Notas), com todas as partes mencionadas implementadas, incluindo:
    - Comentários explicativos.
    - Declaração de variáveis com os modificadores de acesso (`extern`, `static`, `const`, etc.).
    - Funções de entrada e saída de dados.
    - Uso de boas práticas de programação.
3. **Reflexão:** Inclua uma breve **reflexão** sobre o uso dos modificadores de acesso no código e como **boas práticas de programação** ajudam a tornar o código mais claro e eficiente.

📋 Rubrica de Avaliação:

Critério	Descrição	Peso (%)
Correção do Código	O código executa corretamente a entrada e saída de dados, e usa modificadores de acesso.	40%
Boas Práticas de Programação	O código segue boas práticas de programação (comentários, indentação, nomes significativos).	30%
Organização e Clareza	O código está bem estruturado, com boa legibilidade e clareza.	20%
Teste e Resultados	O programa foi testado adequadamente e produziu os resultados esperados.	10%

🎯 Resumo da Atividade:

- **Objetivo:** Ensinar a **linguagem C** com ênfase em **modificadores de acesso**, **boas práticas de programação** e **entrada/saída de dados**.
- **Conceitos:** Uso de **variáveis globais**, **variáveis locais**, **constantes**, **modificadores como `static`, `extern`, `const`, `volatile`, etc.**
- **Ferramenta:** Utilização de **Bloco de Notas** para entrega do código.

Essa atividade ajudará os alunos a entender os conceitos fundamentais de **programação em C** e a **aplicar boas práticas** no desenvolvimento de programas mais robustos e eficientes.